

Kundsegmentering med RFM och K-Means

Projektuppgift del 2

Amir Hamza Jafari, Andreas Malmgren och Robin Hellgren

GitHub: <https://github.com/amirhamza247/ai-customer-segmentation>

1. Bakgrund

Vi ville göra något som ett riktigt företag skulle kunna ha nytta av, och valde därför kundsegmentering. Tanken är att ett företag ska kunna dela in sina kunder i grupper, till exempel kunder som handlar ofta och mycket, kunder som bara handlat en gång och kunder som inte kommit tillbaka på länge. Då kan man rikta kampanjer till rätt grupp.

Vi använder datasetet Online Retail II från Kaggle. Det innehåller transaktioner från en brittisk webbutik mellan december 2009 och december 2011, drygt en miljon rader. Varje rad är en produkt på en faktura, med fakturanummer, antal, datum, pris och kund-ID. Det är just de uppgifterna som behövs för en RFM-analys, så vi tyckte att datan räckte för det vi ville göra.

RFM står för Recency, Frequency och Monetary. Recency är hur många dagar sedan kunden handlade senast, Frequency är hur många olika fakturor kunden har och Monetary är hur mycket kunden har handlat för totalt. Utifrån de tre värdena delar vi in kunderna i grupper med K-Means. Datan har inga färdiga etiketter som säger vilken typ av kund någon är, så det blir oöverskådlig inläring. K-Means är också ganska lätt att förklara, vilket var viktigt för oss.

Vi bestämde tidigt att vi skulle göra en proof of concept och fokusera på att hela flödet fungerar, från att man laddar upp en fil till att man ser segmenten. Därför stödjer appen bara ett CSV-format och vi har inte gjort någon churn- eller LTV-modell.

2. Hur appen fungerar

Användaren laddar upp CSV-filen i Streamlit, och det första appen gör är att validera och rensa den. Vi tar bort rader som saknar kund-ID, makulerade fakturor (de som börjar på C), rader med ogiltigt datum och rader där antal eller pris saknas eller är noll eller negativt. Appen visar hur många rader som togs bort och varför. Dubletter behåller vi, eftersom samma rad kan vara ett riktigt köp två gånger.

Därefter räknas RFM ut för varje kund. Som referensdatum använder vi dagen efter sista köpet i datan, annars skulle resultatet ändras beroende på vilken dag man kör appen. Innan värdena går in i K-Means behöver de förberedas. Frequency och Monetary är väldigt sneda eftersom några få kunder har handlat för enorma summor, så vi gör log1p på dem. Sedan skalar vi alla tre med StandardScaler, så att Monetary inte tar över bara för att siffrorna är större.

Nästa steg är att välja antal kluster. Appen testar K från 2 till 8, räknar silhouette score för varje och föreslår det K som fick högst poäng. Användaren väljer sedan själv 2, 3 eller 4. K-Means tränas med det valda K, och för varje kluster visas antal kunder och medelvärden för Recency, Frequency och Monetary i vanliga enheter. Varje kluster får också en enkel beskrivning som säger om det ligger över eller under snittet, till exempel "More recent / Higher frequency / Higher spending". Till sist visas kunderna i två spridningsdiagram med Recency mot Monetary, ett med vanlig och ett med logaritmisk skala.

Om man har en API-nyckel kan man också be en AI-modell via Groq att förklara klustren i text. Vi skickar bara sammanfattningstabellen och inga kund-ID:n. Procentandelarna räknar vi ut själva i Python innan, eftersom språkmodeller ofta räknar fel.

3. Resultat

Datasetet hade 1 067 371 rader och efter rensningen var 805 549 kvar. Det mesta som försvann var rader utan kund-ID (243 007 st). Resten var 18 744 makuleringar och 71 rader med pris 0. Kvar blev 5 878 kunder.

Att datan är sned syns tydligt. Mediankunden har handlat 3 gånger för ungefär 900 pund, medan den största kunden har handlat för över 600 000 pund. Utan log-transformeringen hade de få största kunderna styrt hela klustringen.

$K = 2$ fick högst silhouette score med 0,419, men $K = 3$ var nästan lika bra med 0,401. För $K = 4$ till 8 låg poängen mellan 0,32 och 0,37. Vi tycker att tre grupper säger mer än två, så vi har främst tittat på $K = 3$.

Det tydligaste resultatet är att en liten del av kunderna står för nästan alla intäkter. Med $K = 3$ hamnar 1 689 kunder (29 %) i ett kluster som handlat nyligen, ofta och mycket. De har i snitt 58 dagar sedan senaste köpet, 15,9 fakturor och 8 691 pund, och tillsammans står de för 83 % av intäkterna. Det största klustret har 2 352 kunder (40 %) som också handlat ganska nyligen, i snitt för 93 dagar sedan, men som köper sällan och för lite pengar, i snitt 2,9 fakturor och 861 pund. De står för 11 % av intäkterna. Det sista klustret har 1 837 kunder (31 %) som inte handlat på länge, i snitt för 474 dagar sedan, med 1,8 fakturor och 566 pund. De står bara för 6 % av intäkterna. Väljer man $K = 4$ delas de bästa kunderna upp ytterligare, och då står 952 kunder (16 %) för 71 % av intäkterna.

Kunderna i det sista klustret har inte handlat på över ett år och har antagligen slutat. Det stora klustret med aktiva kunder som köper sällan är nog där företaget har mest att vinna, eftersom de fortfarande handlar men skulle kunna handla mer.

Resultatet har också sina begränsningar. En silhouette score på runt 0,4 betyder att klustren går att skilja åt men att de överlappar en del, vilket är ganska normalt för kunddata. Beskrivningarna säger bara om ett kluster ligger över eller under snittet och inte hur mycket. Därför har vi valt att inte ge klustren namn som "VIP" eller "lojala kunder", eftersom det kan låta säkrare än det är.

4. Teknik

Vi har skrivit allt i Python 3.14 och använder uv för att hantera paket, så att alla i gruppen får samma versioner. Inläsning, rensning och RFM görs med pandas och NumPy. För maskininläringen använder vi scikit-learn, närmare bestämt StandardScaler, KMeans och silhouette_score. Gränssnittet är gjort i Streamlit. AI-förklaringen går via Groq, och API-nyckeln ligger i en fil som inte laddas upp till GitHub. Koden hålls snygg med Ruff och Black, och vi har använt Git och GitHub för versionshantering.

Koden ligger i tre filer. segmentation.py har alla funktioner för rensning, RFM, skalning och klustring, main.py är Streamlit-appen och ai_analysis.py sköter anropet till AI-modellen. Vi ville hålla isär logiken och gränssnittet så att det är lättare att förstå vad som händer var.

Uppgiften säger att datan ska lagras i en databas, och det gjorde vi i vår första version. Där hade vi ett skript som läste in CSV-filen till SQLite, och sedan räknades RFM ut med SQL-frågor. Den tränade modellen och segmenten sparades också i en databas som Streamlit-appen läste från.

När vi sedan ändrade appen så att man laddar upp sin egen fil kom vi fram till att databasen mest var i vägen. Varje uppladdning analyseras för sig, så det fanns inget som behövde sparas mellan körningarna och databasen blev bara en kopia av CSV-filen. Man var dessutom tvungen att köra fyra skript i rätt ordning innan appen fungerade, och det var lätt hänt att datan i databasen inte längre stämde med koden. Databasfilerna låg också i Git, där de inte går att läsa i en pull request. Därför görs allt i minnet med pandas nu. Om vi senare vill spara resultat, till exempel för att jämföra segment över tid, är det då vi skulle lägga tillbaka SQLite.

En stor del av koden har vi skrivit med hjälp av AI-kodagenter, Claude Code och Codex. För att koden inte skulle bli för stor och krånglig skrev vi en fil med regler för agenten, först CLAUDE.md och sedan AGENTS.md. Där står bland annat att koden ska vara så enkel som möjligt, att inga nya paket får läggas till utan anledning och att agenten ska förklara det den gör. Vi har försökt se till att vi förstår all kod vi lämnar in.

5. Utvärdering av vårt arbete

Det som gick bra var att vi ganska snabbt fick ett flöde som fungerade hela vägen, vilket var målet med PoC:n. Vi delade upp arbetet i små steg och fick varje del att fungera innan vi gick vidare. Vi hade

också standups som vi skrev ner i MEETINGS.md, så att alla visste vem som gjorde vad.

Vi har lärt oss mycket om hur man förbereder data för klustring, till exempel varför man behöver log-transformera och skala, och att silhouette score bara är en fingervisning. Vi har också lärt oss att klustrens nummer inte betyder något i sig, utan att man måste titta på siffrorna för att förstå vad varje kluster är. En annan stor sak var att jobba med AI-agenter. De skriver kod väldigt snabbt, men utan regler blir det lätt mer kod än vad som behövs. Vi lärde oss att granska diffen innan varje commit och att fråga agenten när vi inte förstod något. Ibland satt vi tillsammans i Live Share och följde när en av oss jobbade med agenten.

I Git och GitHub jobbade vi med en branch per person och uppgift, och slog ihop dem med pull requests. Första dagen testade vi hela PR-flödet med några testfiler så att alla visste hur det gick till. Det som fungerade sämre var att två olika versioner av appen byggdes samtidigt på olika brancher, en med SQLite-skript och en med uppladdning. Det gjorde att vi fick börja om lite och lägga tid på att slå ihop dem. Vi skrev också commit-meddelanden på både svenska och engelska, vilket gör historiken lite rörig.

Om vi gjorde om projektet skulle vi bestämma arkitekturen tillsammans innan vi började koda, så att vi inte bygger två versioner parallellt. Vi skulle också komma överens om hur vi jobbar med brancher, commits och code review från början, eftersom code review-delen i MEETINGS.md nästan aldrig användes. Det hade också varit bra att skriva några enkla tester för rensningen och RFM tidigt, och att lägga upp appen på Streamlit Community Cloud så att den får en egen länk.